Übungsskript: Objektorientierte Campus-Verwaltung, GUI-Entwicklung & Datenpersistenz mit Python

Modul: Coding und Digitalisierung (Vorlesung 004/005) – Abstrakte Klassen, Vererbung, Polymorphismus, Tkinter-GUI und JSON-Persistenz

Wintersemester 2026, P. Kern

29. Oktober 2026

Inhaltsverzeichnis

1 Einführung und organisatorische Hinweise

Willkommen zum dritten großen Übungsskript des Moduls „Coding und Digitalisierung“. Nachdem Sie im zweiten Übungsblatt gelernt haben, wie man Sensordaten mit Funktionen, `try/except`, NumPy, Pandas und ersten eigenen Klassen (`class Sensor`) auswertet, vollziehen wir in diesem Arbeitsblatt den Schritt hin zu einer vollständigen, interaktiven **Desktop-Applikation mit mehrstufiger Klassenarchitektur**.

Das vorliegende Dokument ist als durchgängiges Software-Engineering-Projekt konzipiert, das Sie Schritt für Schritt bearbeiten können. Jedes Modul baut logisch auf dem vorherigen auf:

1. **Teil 1 (Module 1–5): Objektorientierte Modellierung & Vererbung.** Sie entwerfen eine abstrakte Basisklasse `Person` für eine Universität und leiten daraus die spezialisierten Kindklassen `Studierender`, `Professor` und `Mitarbeiter` (Staff) ab. Dabei setzen Sie das Prinzip des **Überschreibens (Method Overriding)** sowie die Python-Varianten des **Überladens (Method & Operator Overloading)** gezielt in der Praxis um.
2. **Teil 2 (Module 6–8): Grafische Benutzeroberfläche mit Tkinter (CRUD) & Persistenz.** Sie erweitern Ihr Backend um eine interaktive Eingabemaske mit der Python-Standardbibliothek `tkinter`. Über diese Maske lassen sich Personen in einer zentralen Liste **anlegen (Create)**, **anzeigen (Read)**, **verändern (Update)** und **löschen (Delete)**. Abschließend implementieren Sie eine JSON-Speicher- und Ladeautomatik, sodass alle Daten beim Schließen des Programms dauerhaft auf der Festplatte gesichert und beim nächsten Programmstart automatisch wiederhergestellt werden.

1.1 Verwendung von KI

Sie können die KI der Technischen Hochschule Köln kostenfrei für Ihre Fragen nutzen: https://chat.kiconnect.nrw/app

Wichtig: Fragen Sie nicht nach der reinen Komplettlösung einer Aufgabe, sondern immer nach einer ausführlichen Erklärung eines Konzepts oder einer Fehlermeldung. Eine bloße Lösung unterstützt das Lernen kaum – erst die nachvollziehbare Erklärung (z. B. warum eine abstrakte Klasse nicht direkt instanziiert werden darf oder wie `super()` den Konstruktor der Elternklasse aufruft) fördert das Verständnis für die Klausur und Ihr Semesterprojekt.

Beispiel zur Vertiefung aus Vorlesung 004:
Frage: Was ist in der objektorientierten Programmierung mit Python der genaue Unterschied zwischen **Überschreiben (Overriding)** und **Überladen (Overloading)**?
Antwort: Obwohl beide Begriffe ähnlich klingen, beschreiben sie zwei völlig unterschiedliche Mechanismen:

- **Überschreiben (Method Overriding):** Findet immer zwischen einer *Elternklasse* und einer *Kindklasse* (Vererbung) statt. Die Kindklasse definiert eine Methode mit exakt demselben Namen wie in der Elternklasse neu (z. B. `berechne_budget()` oder `get_rolle()`), um das allgemeine Verhalten durch ein spezialisiertes Verhalten zu ersetzen.
- **Überladen (Overloading):** Bedeutet im klassischen Sinne (wie in Java oder C++), innerhalb *derselben Klasse* mehrere gleichnamige Methoden mit unterschiedlichen Parameterlisten anzulegen. Da Python dynamisch typisiert ist, würde eine zweite `def`-Definition gleichen Namens die erste einfach überschreiben. In Python realisiert man das **Methoden-Überladen** daher über:
  1. *Default-Parameter und Typweichen* (`isinstance(x, list)`) oder variable Argumentlisten (`*args`), sodass eine einzige Methode unterschiedlich viele oder verschiedenartige Übergabewerte verarbeiten kann.
  2. *Operator-Überladung (Dunder-Methoden)* wie `__str__` oder `__eq__`, mit denen bestehende Python-Operatoren (wie `==` oder `print()`) für eigene Klassen mit neuer Bedeutung belegt werden.

1.2 Das Projekt: Digitales Campus-Management-System „TH-CampusAdmin“

Die Hochschulleitung möchte die bisherige, fehleranfällige Excel-Verwaltung aller Hochschulangehörigen durch eine moderne, objektorientierte Python-Anwendung mit grafischer Oberfläche ablösen. An der Universität gibt es drei unterschiedliche Personengruppen:

1. **Studierende (`Studierender`):** Besitzen einen Studiengang, ein aktuelles Fachsemester sowie ein ECTS-Konto und schreiben sich für Module ein.
2. **Professorinnen und Professoren (`Professor`):** Gehören einem Lehrstuhl/Fachgebiet an, verfügen über ein Forschungsbudget und verwalten eine dynamische Liste eigener Vorlesungen.
3. **Verwaltungs- und Technikpersonal (`Mitarbeiter` / Staff):** Arbeiten in einer Abteilung (z. B. Prüfungsamt, IT-Service, Laborbetrieb), haben eine Wochenarbeitszeit sowie eine Tarifgruppe.

Alle drei Gruppen sind Personen an der Universität (sie teilen sich Attribute wie `person_id`, `vorname`, `nachname` und `email`), gleichzeitig existiert aber keine „allgemeine Person“ ohne konkrete Rolle auf dem Campus. Deshalb muss die Basisklasse `Person` zwingend als **abstrakte Klasse** entworfen werden.

1.3 Lernziele

- Entwurf einer abstrakten Basisklasse mittels `from abc import ABC, abstractmethod`, um Schnittstellenverträge in Klassenhierarchien zu erzwingen.
- Sichere Anwendung der Vererbung (`class Kind(Eltern):`) und Konstruktor-Verkettung mit `super().__init__(...)`.
- Gezielter Einsatz von **Method Overriding** (Überschreiben geerbter und abstrakter Methoden) und Polymorphismus bei der Iteration über heterogene Objektlisten.
- Umsetzung von **Method Overloading** in Python über Default-Parameter, `*args` und Typprüfung (`isinstance()`) sowie **Operator Overloading** (`__str__`, `__eq__`).
- Aufbau einer modularen Drei-Schichten-Architektur (Datenmodell `models.py`, Datenhaltung `storage.py`, Benutzeroberfläche `gui_app.py`).
- Entwicklung einer interaktiven **Tkinter-GUI** mit Tabellenansicht (`ttk.Treeview`), dynamischen Eingabefeldern und Ereignisbehandlung (Event-Binding zum Anlegen, Ändern und Löschen von Personen).
- Implementierung einer fehlertoleranten **JSON-Persistenz** (Serialisierung von Objekten in Dictionaries beim Speichern und automatische Rekonstruktion beim Programmstart).

2 Modul 1: Projekt-Setup und die abstrakte Basisklasse `Person`

In einem gut strukturierten Softwareprojekt trennen wir die Datenklassen (das sogenannte *Backend-Modell*) sauber von der späteren grafischen Oberfläche. Wir beginnen mit der Datei `models.py` und entwerfen das abstrakte Fundament unserer Universität.

2.1 Aufgabe 1.1: Projektordner und Dateistruktur anlegen

Erstellen Sie auf Ihrem Computer einen neuen Ordner mit dem Namen `CampusVerwaltung` und öffnen Sie diesen in Visual Studio Code. Legen Sie darin folgende drei Python-Dateien an:

- `models.py` (hier entstehen in Modul 1 bis 4 unsere Klassen `Person`, `Studierender`, `Professor` und `Mitarbeiter`)
- `storage.py` (hier entsteht in Modul 5 die Listenverwaltung und das Speichern/Laden als JSON)
- `gui_app.py` (hier entsteht in Modul 6 bis 8 unsere grafische Tkinter-Anwendung)

Hinweis: Da sowohl `abc`, `json` als auch `tkinter` direkt in der Python-Standardbibliothek („Batteries Included“) enthalten sind, müssen Sie über `pip` keinerlei externe Pakete nachinstallieren!

2.2 Aufgabe 1.2: Warum muss `Person` abstrakt sein? (`ABC` & `@abstractmethod`)

An unserer Universität hat jede Person eine eindeutige ID (z. B. `"P-1001"`), einen Vornamen, einen Nachnamen und eine E-Mail-Adresse. Dennoch gibt es auf dem Campus niemanden, der *nur* ein nacktes Objekt vom Typ `Person` ist – jeder Angehörige ist konkret entweder Studierender, Professor oder Mitarbeiter.

Um in Python technisch zu verhindern, dass jemand versehentlich `p = Person(...)` aufruft, nutzen wir das Modul `abc` (*Abstract Base Classes*):

1. Öffnen Sie `models.py` und importieren Sie am Dateianfang: `from abc import ABC, abstractmethod`
2. Definieren Sie die Klasse `class Person(ABC):`, die von `ABC` erbt.
3. Implementieren Sie den Konstruktor `__init__(self, person_id, vorname, nachname, email)`:
   - Prüfen Sie zur Sicherheit, ob `vorname` oder `nachname` leer sind (falls ja: `raise ValueError("Vor- und Nachname dürfen nicht leer sein!")`).
   - Speichern Sie alle vier Parameter als Objektattribute (`self.person_id`, `self.vorname`, `self.nachname`, `self.email`).
4. Fügen Sie eine reguläre Methode `voller_name(self)` hinzu, die den String `f"{self.vorname} {self.nachname}"` zurückgibt.
5. Definieren Sie nun **drei abstrakte Methoden** mit dem Dekorator `@abstractmethod`, deren Rumpf lediglich aus dem Schlüsselwort `pass` besteht:
   - `get_rolle(self)`: Soll später die genaue Rollenbezeichnung als String liefern (z. B. `"Professor"`).
   - `get_details(self)`: Soll später eine rollenspezifische Zusammenfassung für die GUI-Tabelle liefern.
   - `to_dict(self)`: Soll später alle Attribute des Objekts als Python-Dictionary für den JSON-Export liefern.

Listing 1: Grundgerüst der abstrakten Klasse Person in models.py

```python
from abc import ABC, abstractmethod

class Person(ABC):
    """Abstrakte Basisklasse für alle Angehörigen der Universität."""

    def __init__(self, person_id: str, vorname: str, nachname: str, email: str):
        if not str(vorname).strip() or not str(nachname).strip():
            raise ValueError("Vor- und Nachname dürfen nicht leer sein!")
        self.person_id = str(person_id).strip()
        self.vorname = str(vorname).strip()
        self.nachname = str(nachname).strip()
        self.email = str(email).strip()

    def voller_name(self) -> str:
        """Gibt den vollständigen Namen der Person zurück."""
        return f"{self.vorname} {self.nachname}"

    @abstractmethod
    def get_rolle(self) -> str:
        """Muss von jeder abgeleiteten Klasse überschrieben werden."""
        pass

    @abstractmethod
    def get_details(self) -> str:
        """Liefert rollenspezifische Zusatzinformationen als Text."""
        pass

    @abstractmethod
    def to_dict(self) -> dict:
        """Konvertiert das Objekt für die JSON-Speicherung in ein Dictionary."""
        pass
```

2.3 Aufgabe 1.3: Das Instanziierungs-Experiment (TypeError prüfen)

Testen Sie den Schutzmechanismus Ihrer abstrakten Klasse am Ende von `models.py` in einem `if __name__ == "__main__":`-Block:

1. Versuchen Sie in einem `try`-Block, direkt ein Objekt der Klasse `Person` zu erzeugen: `test_p = Person("P-01", "Erika", "Muster", "erika@th-koeln.de")`
2. Fangen Sie den entstehenden `TypeError` mit `except TypeError as e:` ab und geben Sie die Original-Fehlermeldung von Python auf der Konsole aus.

Erwartetes Ergebnis im Terminal:

```
=== Modul 1: Test der abstrakten Basisklasse Person ===
Erwarteter Schutz-Fehler abgefangen:
Can't instantiate abstract class Person without an implementation for abstract
    methods 'get_details', 'get_rolle', 'to_dict'
```

2.4 Kontrollfragen zu Modul 1

1. Welche zwei Bedingungen müssen in Python gleichzeitig erfüllt sein, damit eine Klasse als echt abstrakt gilt und nicht instanziiert werden kann?
2. Warum schreiben wir in den abstrakten Methoden `get_rolle(self)` und `get_details(self)` innerhalb der Klasse `Person` nur das Schlüsselwort `pass`?
3. Was passiert, wenn Sie später eine Kindklasse `class Gastdozent(Person):` anlegen, dort aber vergessen, die Methode `to_dict(self)` zu programmieren?

3 Modul 2: Konkrete Personenklassen ableiten (`Studierender`, `Professor`, `Mitarbeiter`)

Nun leiten wir aus unserer abstrakten Basisklasse `Person` die drei konkreten Personengruppen der Universität ab. Jede Kindklasse ruft über `super().__init__(...)` den Konstruktor der Elternklasse auf, ergänzt eigene Attribute und eigene Fachmethoden und implementiert (überschreibt) die drei geforderten abstrakten Methoden.

3.1 Aufgabe 2.1: Die Klasse `Studierender` implementieren

Erweitern Sie Ihre Datei `models.py` um die Klasse `Studierender(Person)`:

1. **Konstruktor:** `__init__(self, person_id, vorname, nachname, email, studiengang, semester=1, ects=0)`
   - Rufen Sie zuerst `super().__init__(person_id, vorname, nachname, email)` auf.
   - Speichern Sie `self.studiengang = str(studiengang)`, `self.semester = int(semester)` und `self.ects = int(ects)`.
2. **Eigene Fachmethode:** `pruefung_bestehen(self, modul_name, punkte)`
   - Prüft, ob `int(punkte) > 0` ist, addiert die Punkte auf `self.ects` und gibt eine Bestätigung auf der Konsole aus.
3. **Pflichtmethoden überschreiben:**
   - `get_rolle(self)` gibt den String `"Studierender"` zurück.
   - `get_details(self)` gibt `f"Studiengang: {self.studiengang} (Sem. {self.semester}, {self.ects} ECTS)"` zurück.
   - `to_dict(self)` gibt ein Dictionary mit allen Attributen zurück (inklusive `"rolle": "Studierender"`, damit wir beim späteren Laden aus der Datei wissen, welche Klasse erzeugt werden muss!).

3.2 Aufgabe 2.2: Die Klasse `Professor` mit Vorlesungsverwaltung

Implementieren Sie als zweite abgeleitete Klasse `class Professor(Person):`:

1. **Konstruktor:** `__init__(self, person_id, vorname, nachname, email, fachgebiet, vorlesungen=`
   - Rufen Sie `super().__init__(person_id, vorname, nachname, email)` auf.
   - Speichern Sie `self.fachgebiet = str(fachgebiet)`.
   - Legen Sie die Vorlesungsliste sicher an: Falls `vorlesungen is None` ist, setzen Sie `self.vorlesungen = []`, andernfalls `self.vorlesungen = list(vorlesungen)`.
2. **Eigene Fachmethode:** `vorlesung_entfernen(self, titel)` entfernt eine Vorlesung aus `self.vorlesungen`, falls sie in der Liste enthalten ist. (Die Methode zum Hinzufügen von Vorlesungen gestalten wir in Modul 3 als überladene Methode!)
3. **Pflichtmethoden überschreiben:**
   - `get_rolle(self)` gibt `"Professor"` zurück.
   - `get_details(self)` verbindet die Vorlesungen mit Komma (`", ".join(self.vorlesungen)` oder `"Keine"`, falls die Liste leer ist) und gibt `f"Lehrstuhl: {self.fachgebiet} | VL: {vl_text}"` zurück.
   - `to_dict(self)` liefert ein Dictionary mit `"rolle": "Professor"`, den Basisdaten, `"fachgebiet"` und der Liste `"vorlesungen"`.

3.3 Aufgabe 2.3: Die Klasse `Mitarbeiter` (University Staff)

Implementieren Sie als dritte abgeleitete Klasse `class Mitarbeiter(Person):`:

1. **Konstruktor:** `__init__(self, person_id, vorname, nachname, email, abteilung, wochenstunden`
   - Rufen Sie `super().__init__(person_id, vorname, nachname, email)` auf.
   - Speichern Sie `self.abteilung = str(abteilung)` und `self.wochenstunden = float(wochenstund`
2. **Eigene Fachmethode:** `aendere_arbeitszeit(self, neue_stunden)` aktualisiert die Wochenstunden nach einer Plausibilitätsprüfung ($0 <$ Stunden ≤ 42).
3. **Pflichtmethoden überschreiben:**
   - `get_rolle(self)` gibt `"Mitarbeiter"` zurück.
   - `get_details(self)` gibt `f"Abteilung: {self.abteilung} ({self.wochenstunden:.1f} Std./Woche)"` zurück.
   - `to_dict(self)` liefert das vollständige Dictionary inklusive `"rolle": "Mitarbeiter"`, `"abteilung"` und `"wochenstunden"`.

Erwartetes Ergebnis bei einem ersten Instanziierungstest in `models.py`:

```
=== Modul 2: Konkrete Kindklassen erfolgreich instanziiert ===
[Studierender] Anna Schmidt | Studiengang: Maschinenbau (Sem. 3, 60 ECTS)
[Professor] Peter Kern | Lehrstuhl: Digitalisierung | VL: Coding 1, Data Science
[Mitarbeiter] Markus Weber | Abteilung: IT-Service (39.5 Std./Woche)
```

3.4 Kontrollfragen zu Modul 2

1. Warum verwenden wir im Konstruktor von `Professor` den Standardwert `vorlesungen=None` und setzen erst im Rumpf `self.vorlesungen = []`, anstatt direkt im Funktionskopf `vorlesungen=[]` zu schreiben? (Stichwort: *Mutable Default Arguments*)
2. Welche Codezeilen sparen wir uns in `Studierender`, `Professor` und `Mitarbeiter` durch den Aufruf von `super().__init__(...)`?
3. Warum speichern wir in `to_dict(self)` zusätzlich das Schlüssel-Wert-Paar `"rolle": self.get_rolle()` ab, obwohl dies gar kein eigenes Attribut im Konstruktor war?

4 Modul 3: Überschreiben (Overriding) und Überladen (Overloading) in der Praxis

In diesem Modul vertiefen wir zwei der wichtigsten Konzepte der Objektorientierung, die auch in Prüfungen regelmäßig abgefragt werden: Das **Überschreiben** von Methoden in einer Vererbungshierarchie und das **Überladen** von Methoden und Operatoren.

4.1 Aufgabe 3.1: Methoden-Überschreiben (Overriding) mit `super()`-Erweiterung

Neben rein abstrakten Methoden besitzt eine Basisklasse oft Methoden mit einem sinnvollen Standardverhalten, das von einigen Kindklassen übernommen, von anderen erweitert und von manchen komplett ersetzt wird.

1. Fügen Sie der Basisklasse `Person` eine Methode `erstelle_campus_ausweis(self)` hinzu, die folgenden Basis-Text zurückgibt: `return f"AUSWEIS [{self.person_id}] {self.voller_name()} ({self.get_rolle()})"`
2. **Erweiterndes Überschreiben in `Professor`:** Überschreiben Sie `erstelle_campus_ausweis(self)` in der Klasse `Professor`. Rufen Sie darin über `basis = super().erstelle_campus_ausweis()` den Text der Elternklasse ab und hängen Sie `f"{basis} - Zutritt: Alle Labore & Senatssaal"` an.
3. **Ersetzendes Überschreiben in `Studierender`:** Überschreiben Sie die Methode in `Studierender` so, dass zusätzlich die Gültigkeit als Semesterticket vermerkt wird: `return f"STUDIERENDENAUSWEIS [{self.person_id}] {self.voller_name()} | {self.studiengang} (inkl. NRW-Ticket)"`
4. In der Klasse `Mitarbeiter` implementieren Sie diese Methode *nicht* neu – sie nutzt automatisch die geerbte Standardversion aus `Person`!

4.2 Aufgabe 3.2: Methoden-Überladen (Method Overloading) in Python umsetzen

Ein Professor soll über die Methode `vorlesung_hinzufuegen(...)` neue Lehrveranstaltungen in seine Liste `self.vorlesungen` eintragen können. In der Praxis möchte man diese Methode jedoch auf **drei unterschiedliche Arten** aufrufen können (klassisches Szenario für *Method Overloading*):

- **Variante 1 (Einzelner String):** `prof.vorlesung_hinzufuegen("Informatik 1")`
- **Variante 2 (Liste aus Strings):** `prof.vorlesung_hinzufuegen(["Robotik", "KI-Grundlagen"])`
- **Variante 3 (Beliebig viele Einzelargumente):** `prof.vorlesung_hinzufuegen("Mathe 1", "Mathe 2", "Statistik")`

Da Python keine drei gleichnamigen `def`-Blöcke untereinander erlaubt, implementieren wir das Überladen über eine Kombination aus variablen Argumenten (`*args`) und Typprüfung (`isinstance()`):

Listing 2: Überladene Methode vorlesung_hinzufuegen in der Klasse Professor

```
    def vorlesung_hinzufuegen(self, *neue_vorlesungen):
        """
        Überladene Methode: Akzeptiert einen einzelnen String, eine Liste/Tupel
        von Strings oder beliebig viele kommagetrennte Strings (*args).
        """
        for eintrag in neue_vorlesungen:
            # Fall A: Es wurde eine Liste oder ein Tupel übergeben
            if isinstance(eintrag, (list, tuple)):
                for einzel_titel in eintrag:
                    titel_clean = str(einzel_titel).strip()
                    if titel_clean and titel_clean not in self.vorlesungen:
```

```
                        self.vorlesungen.append(titel_clean)
            # Fall B: Es wurde ein einzelner String übergeben
            elif isinstance(eintrag, str):
                titel_clean = eintrag.strip()
                if titel_clean and titel_clean not in self.vorlesungen:
                    self.vorlesungen.append(titel_clean)
```

Zweites Überladungs-Beispiel (Default-Parameter in `Studierender`): Implementieren Sie in der Klasse `Studierender` eine überladene Methode `studienfortschritt_berechnen(self, ziel_ects=180, zeige_details=False)`:

- Wird sie ohne Argumente aufgerufen (`stud.studienfortschritt_berechnen()`), berechnet sie den prozentualen Fortschritt für einen Bachelor-Studiengang (180 ECTS) und gibt nur die Kommazahl zurück (z. B. `33.3`).
- Wird sie mit `ziel_ects=120` (Master) und `zeige_details=True` aufgerufen, gibt sie stattdessen einen formatierten Textbericht zurück: `"Fortschritt: 60 / 120 ECTS (50.0%)"`.

4.3 Aufgabe 3.3: Operator-Überladung (Dunder-Methoden `__str__` und `__eq__`)

In Python lassen sich auch eingebaute Operatoren wie `print(obj)` oder der Gleichheitsoperator `obj1 == obj2` überladen:

1. Überladen Sie in der Basisklasse `Person` die Methode `__str__(self)`: `return f"[{self.person_id}] {self.voller_name()} ({self.get_rolle()}) - {self.email} | {self.get_details()}"`
2. Überladen Sie in `Person` den Gleichheitsoperator `__eq__(self, other)`: Zwei Personen-Objekte an der Universität sollen genau dann als gleich (`True`) gelten, wenn `isinstance(other, Person)` erfüllt ist und beide exakt dieselbe `person_id` besitzen (`self.person_id == other.person_id`).

Erwartetes Ergebnis im Terminal beim Testen von Modul 3:

```
=== Modul 3: Überschreiben (Overriding) & Überladen (Overloading) ===
--- 1. Test Method Overriding (Ausweise) ---
STUDIERENDENAUSWEIS [S-101] Anna Schmidt | Maschinenbau (inkl. NRW-Ticket)
AUSWEIS [P-201] Peter Kern (Professor) - Zutritt: Alle Labore & Senatssaal
AUSWEIS [M-301] Markus Weber (Mitarbeiter)

--- 2. Test Method Overloading (Vorlesungen hinzufügen) ---
Nach Einzel-String: ['Coding und Digitalisierung']
Nach Liste:         ['Coding und Digitalisierung', 'Robotik', 'KI-Systeme']
Nach *args:         ['Coding und Digitalisierung', 'Robotik', 'KI-Systeme', 'Python 2
    ', 'Datenbanken']

--- 3. Test Method Overloading (Studienfortschritt) ---
Aufruf 1 (Standard Bachelor): 33.3%
Aufruf 2 (Master + Details):  Fortschritt: 60 / 120 ECTS (50.0%)

--- 4. Test Operator Overloading (__eq__) ---
Vergleich S-101 mit S-101 (Kopie): True
Vergleich S-101 mit P-201:         False
```

4.4 Kontrollfragen zu Modul 3

1. Warum führt in Python eine zweite Methodendefinition `def hinzufuegen(self, a, b):` unterhalb von `def hinzufuegen(self, a):` in derselben Klasse nicht zu echtem Methoden-Überladen wie in C++ oder Java?
2. Wie hilft uns die Operator-Überladung `__eq__(self, other)` später dabei, wenn wir mit `if neue_person in personen_liste:` prüfen wollen, ob eine Personen-ID bereits vergeben ist?

3. Welchen Vorteil hat es, beim Überschreiben einer Methode in der Kindklasse trotzdem `super().methodenname` aufzurufen?

5 Modul 4: Polymorphismus und zentrale Listenverwaltung

Bevor wir die grafische Oberfläche und das Dateisystem anbinden, erstellen wir in der Datei `storage.py` eine Verwaltungs-Klasse `UniversitaetsVerwaltung`. Diese hält alle angelegten Personen-Objekte (egal ob Studierende, Professoren oder Mitarbeiter) gemeinsam in einer einzigen Python-Liste `self.personen = []` und stellt Methoden zum Anlegen, Suchen, Ändern und Löschen bereit.

5.1 Aufgabe 4.1: Die Klasse `UniversitaetsVerwaltung` in `storage.py` anlegen

Öffnen Sie die Datei `storage.py` und importieren Sie Ihre Klassen aus `models.py`:

Listing 3: Grundstruktur der Listenverwaltung in storage.py

```
import json
import os
from models import Person, Studierender, Professor, Mitarbeiter

class UniversitaetsVerwaltung:
    """Verwaltet die Liste aller Personen an der Universität inkl. JSON-Persistenz."""

    def __init__(self, dateipfad: str = "campus_daten.json"):
        self.dateipfad = dateipfad
        self.personen: list[Person] = []
```

Implementieren Sie nun die vier grundlegenden Verwaltungsoperationen (**CRUD**: Create, Read, Update, Delete) auf der Liste `self.personen`:

1. **Person suchen (`finde_nach_id(self, person_id)`):** Durchläuft `self.personen` in einer Schleife. Wenn `p.person_id == person_id` gefunden wird, gibt die Methode das Objekt `p` zurück, andernfalls `None`.

2. **Person hinzufügen (`person_hinzufuegen(self, neue_person)`):**

   - Prüft zuerst mit `if neue_person in self.personen:` (nutzt automatisch unser überladenes `__eq__` aus Modul 3!), ob die ID bereits existiert. Falls ja, wirft sie einen `ValueError(f"ID '{neue_person.person_id}' existiert bereits!")`.
   - Hängt `neue_person` mit `self.personen.append(neue_person)` an die Liste an.

3. **Person verändern / ersetzen (`person_aktualisieren(self, aktualisierte_person)`):**

   - Sucht den Index `i` in `self.personen`, an dem `p.person_id == aktualisierte_person.person_id` steht, und ersetzt den Listeneintrag durch `self.personen[i] = aktualisierte_person`.
   - Wurde die ID nicht gefunden, wird ein `KeyError` ausgelöst.

4. **Person löschen (`person_loeschen(self, person_id)`):**

   - Sucht die Person über `finde_nach_id(person_id)`. Wenn sie gefunden wurde, wird sie mit `self.personen.remove(gefunden)` aus der Liste entfernt und `True` zurückgegeben (sonst `False`).

5.2 Aufgabe 4.2: Polymorphismus in Aktion – Gemeinsame Auswertung heterogener Objekte

Dank **Polymorphismus** (Vielgestaltigkeit) können wir über die gemischte Liste `self.personen` iterieren und auf jedem Element `p.get_rolle()` und `p.get_details()` aufrufen, ohne mit `if/else` prüfen zu müssen, um welche konkrete Klasse es sich handelt.

1. Schreiben Sie in `UniversitaetsVerwaltung` eine Methode `zeige_alle_personen(self)`, die alle gespeicherten Objekte nummeriert im Terminal ausgibt.

2. Schreiben Sie zusätzlich eine Statistik-Methode `zaehle_rollen(self)`, die ein Dictionary `{"Studierender": ..., "Professor": ..., "Mitarbeiter": ...}` mit der aktuellen Anzahl je Personengruppe zurückgibt.

5.3 Kontrollfragen zu Modul 4

1. Warum funktioniert der Aufruf `p.get_details()` in einer Schleife über `self.personen` für jedes Objekt fehlerfrei, obwohl `Studierender`, `Professor` und `Mitarbeiter` völlig unterschiedliche Attribute besitzen?

2. Welchen Vorteil bietet unsere zentrale Methode `person_aktualisieren()` gegenüber dem direkten Überschreiben einzelner Attribute aus der GUI heraus, insbesondere wenn sich sogar die Rolle einer Person ändert (z. B. ein Studierender wird nach dem Abschluss zum Mitarbeiter)?

6 Modul 5: Datenpersistenz – Speichern und Laden beim Programmstart (JSON)

Solange unsere Personen-Objekte nur in der Liste `self.personen` im Arbeitsspeicher (RAM) liegen, gehen alle Eingaben verloren, sobald das Programm beendet wird. In diesem Modul rüsten wir `UniversitaetsVerwaltung` mit einer dauerhaften Speicherung im menschenlesbaren **JSON-Format** (*JavaScript Object Notation*) aus.

6.1 Aufgabe 5.1: Serialisierung – Die Liste als JSON-Datei speichern

Implementieren Sie in der Klasse `UniversitaetsVerwaltung` (in `storage.py`) die Methode `speichern(self)`:

1. Erzeugen Sie eine Liste von Dictionaries, indem Sie für jedes Objekt `p` in `self.personen` dessen Methode `p.to_dict()` aufrufen: `daten_liste = [p.to_dict() for p in self.personen]`
2. Öffnen Sie die Zieldatei `self.dateipfad` sicher mit einem Context-Manager (`with open(self.dateipfad, "w", encoding="utf-8") as f:`) im Schreibmodus.
3. Schreiben Sie die Liste mit `json.dump(daten_liste, f, indent=4, ensure_ascii=False)` formatiert in die Datei.

6.2 Aufgabe 5.2: Deserialisierung (Factory-Methode) – Objekte beim Programmstart rekonstruieren

Beim Laden der JSON-Datei liefert `json.load(f)` zunächst nur gewöhnliche Python-Dictionaries zurück. Wir müssen aus jedem Dictionary wieder das passende echte Python-Objekt (`Studierender`, `Professor` oder `Mitarbeiter`) erzeugen. Dafür schreiben wir eine sogenannte **Factory-Methode** (Fabrikmethode):

Listing 4: Speichern und Laden mit Objekt-Rekonstruktion in storage.py

```
    @staticmethod
    def objekt_aus_dict(d: dict) -> Person:
        """Fabrikmethode: Rekonstruiert aus einem JSON-Dictionary das passende Objekt."""
        rolle = d.get("rolle", "")
        if rolle == "Studierender":
            return Studierender(
                person_id=d["person_id"],
                vorname=d["vorname"],
                nachname=d["nachname"],
                email=d["email"],
                studiengang=d.get("studiengang", "Unbekannt"),
                semester=d.get("semester", 1),
                ects=d.get("ects", 0)
            )
        elif rolle == "Professor":
            return Professor(
                person_id=d["person_id"],
                vorname=d["vorname"],
                nachname=d["nachname"],
                email=d["email"],
                fachgebiet=d.get("fachgebiet", "Allgemein"),
                vorlesungen=d.get("vorlesungen", [])
            )
        elif rolle == "Mitarbeiter":
            return Mitarbeiter(
                person_id=d["person_id"],
                vorname=d["vorname"],
                nachname=d["nachname"],
```

```
                email=d["email"],
                abteilung=d.get("abteilung", "Verwaltung"),
                wochenstunden=d.get("wochenstunden", 40.0)
            )
        else:
            raise ValueError(f"Unbekannte Rolle in JSON-Datei: '{rolle}'")

    def laden(self) -> bool:
        """Lädt die Personenliste beim Programmstart aus der JSON-Datei."""
        if not os.path.exists(self.dateipfad):
            return False
        try:
            with open(self.dateipfad, "r", encoding="utf-8") as f:
                roh_liste = json.load(f)
            self.personen = [self.objekt_aus_dict(eintrag) for eintrag in roh_liste]
            return True
        except (json.JSONDecodeError, KeyError, ValueError) as e:
            print(f"[Warnung] Fehler beim Laden der Datei '{self.dateipfad}': {e}")
            return False
```

6.3 Aufgabe 5.3: Automatische Beispieldaten bei leerem Erststart

Ergänzen Sie eine Hilfsmethode `erzeuge_beispiel_daten(self)`, die aufgerufen wird, falls beim ersten Programmstart noch keine Datei `campus_daten.json` existiert. Legen Sie darin mindestens zwei Studierende, zwei Professoren (mit Vorlesungen) und zwei Mitarbeiter an und rufen Sie direkt danach `self.speichern()` auf.

Blick in die erzeugte Datei `campus_daten.json`:

```
[
    {
        "rolle": "Professor",
        "person_id": "P-101",
        "vorname": "Peter",
        "nachname": "Kern",
        "email": "peter.kern@th-koeln.de",
        "fachgebiet": "Coding & Digitalisierung",
        "vorlesungen": ["Informatik 1", "Coding und Digitalisierung"]
    },
    {
        "rolle": "Studierender",
        "person_id": "S-201",
        "vorname": "Anna",
        "nachname": "Schmidt",
        "email": "anna.schmidt@smail.th-koeln.de",
        "studiengang": "Erneuerbare Energien",
        "semester": 3,
        "ects": 60
    }
]
```

6.4 Kontrollfragen zu Modul 5

1. Warum kann `json.dump()` ein benutzerdefiniertes Python-Objekt wie `Professor` nicht direkt ohne unsere Hilfsmethode `to_dict()` speichern?
2. Welchen Zweck erfüllt der Parameter `ensure_ascii=False` in `json.dump()`, wenn unsere Vorlesungstitel oder Namen deutsche Umlaute (ä, ö, ü, ß) enthalten?
3. Warum fangen wir in `laden()` gezielt den Fehler `json.JSONDecodeError` ab? Wann könnte dieser in der Praxis auftreten?

7 Modul 6: Schritt 2 – Grafische Benutzeroberfläche mit Tkinter aufbauen

Nachdem unser objektorientiertes Backend (`models.py`) und die Datenpersistenz (`storage.py`) zuverlässig arbeiten, entwickeln wir im zweiten großen Schritt in der Datei `gui_app.py` eine interaktive Desktop-Oberfläche mit **Tkinter**.

7.1 Aufgabe 6.1: Fenster-Architektur und Layout-Bereiche verstehen

Eine übersichtliche Verwaltungs-Maske gliedert sich in drei visuelle Bereiche (`ttk.LabelFrame`):

1. **Oben – Eingabe- und Bearbeitungsmaske:** Enthält eine Dropdown-Auswahl (`ttk.Combobox`) für die Rolle (`Studierender`, `Professor`, `Mitarbeiter`), Eingabefelder (`ttk.Entry`) für die allgemeinen Personendaten (`ID`, `Vorname`, `Nachname`, `E-Mail`) sowie dynamische Zusatzfelder für die jeweilige Rolle (z. B. Vorlesungsliste beim Professor oder Studiengang/Semester beim Studierenden).
2. **Mitte – Aktionsleiste (Buttons):** Schaltflächen für **„Neu anlegen“**, **„Auswahl speichern (Ändern)“**, **„Auswahl löschen“**, **„Eingabefelder leeren“** sowie **„Daten jetzt sichern (JSON)“**.
3. **Unten – Tabellenübersicht (`ttk.Treeview`):** Zeigt alle aktuell in der Liste gespeicherten Personen übersichtlich mit den Spalten `ID`, `Rolle`, `Name`, `E-Mail` und `Details` an. Klickt man eine Zeile in der Tabelle an, werden deren Daten automatisch zurück in die oberen Eingabefelder geladen!

7.2 Aufgabe 6.2: Dynamische Beschriftung der Zusatzfelder je nach Rolle

Da jede unserer drei abgeleiteten Personenklassen unterschiedliche Zusatzattribute besitzt, passen wir die Beschriftung der drei rollenspezifischen Eingabefelder (`Feld 1`, `Feld 2`, `Feld 3`) automatisch an, sobald der Benutzer die Rolle in der Combobox umschaltet:

Gewählte Rolle	Zusatzfeld 1	Zusatzfeld 2	Zusatzfeld 3
`Studierender`	Studiengang (z. B. Informatik)	Fachsemester (z. B. 3)	ECTS-Punkte (z. B. 60)
`Professor`	Fachgebiet / Lehrstuhl	Vorlesungen (kommagetrennt)	*(nicht benötigt / deaktiviert)*
`Mitarbeiter`	Abteilung (z. B. IT-Service)	Wochenstunden (z. B. `39.5`)	*(nicht benötigt / deaktiviert)*

Tabelle 1: Dynamische Zuordnung der Eingabefelder in der Tkinter-Maske

7.3 Kontrollfragen zu Modul 6

1. Warum kapseln wir auch unsere Tkinter-Oberfläche in einer eigenen Klasse `class CampusApp:` statt alle Buttons und Eingabefelder als globale Variablen im Skript anzulegen?
2. Welches Tkinter-Ereignis (Event) wird ausgelöst, wenn der Benutzer in einer `ttk.Combobox` einen anderen Eintrag auswählt, und wie verknüpft man dieses per `.bind()` mit einer eigenen Methode?

8 Modul 7: CRUD-Aktionen in der Maske und Komplettcode

In diesem Modul verknüpfen wir die Schaltflächen unserer Tkinter-Maske mit der Liste in `UniversitaetsVerwaltung` und sorgen dafür, dass die Daten beim Programmstart geladen und bei jeder Änderung (sowie beim Schließen des Fensters) automatisch gespeichert werden.

8.1 Aufgabe 7.1: Auslesen der Maske und Erzeugen des passenden Objekts

Schreiben Sie in Ihrer GUI-Klasse `CampusApp` eine Hilfsmethode `_baue_objekt_aus_maske(self) -> Person`:

1. Lesen Sie die Textwerte der Eingabefelder über `.get()` aus.
2. Je nachdem, welche Rolle aktuell in der Combobox ausgewählt ist (`"Studierender"`, `"Professor"` oder `"Mitarbeiter"`), instanziieren Sie das entsprechende Objekt.
3. **Besonderheit bei `Professor`:** Nutzen Sie Ihre in Modul 3 überladene Methode `vorlesung_hinzufuegen()`, indem Sie den kommagetrennten String aus Zusatzfeld 2 mit `[v.strip() for v in feld2.split(",") if v.strip()]` in eine Liste zerlegen und übergeben!
4. Fangen Sie Fehleingaben (z. B. Buchstaben im Feld „Fachsemester“ oder leere Pflichtfelder) sauber mit `try / except ValueError` ab und zeigen Sie dem Benutzer ein informatives Dialogfenster mittels `messagebox.showerror("Eingabefehler", str(e))` an.

8.2 Aufgabe 7.2: Anlegen, Ändern, Löschen und Laden bei Programmstart

Verbinden Sie die vier Hauptfunktionen der Oberfläche:

1. **Programmstart (`__init__`):** Rufen Sie direkt beim Start `if not self.verwaltung.laden(): self.verwaltung.erzeuge_beispiel_daten()` auf und füllen Sie die Tabelle (`self.aktualisiere_tabell`
2. **Neu anlegen (`aktion_anlegen`):** Baut das Objekt aus der Maske, ruft `self.verwaltung.person_hinzufue` sowie `self.verwaltung.speichern()` auf und aktualisiert die Tabelle.
3. **Auswahl in Maske laden (`on_tabellen_klick`):** Klickt der Nutzer eine Zeile in der `Treeview`-Tabelle an (`<<TreeviewSelect>>`), wird die ausgewählte `person_id` ermittelt, das Objekt über `finde_nach_id()` geholt und alle Eingabefelder der Maske mit dessen Attributen befüllt.
4. **Änderung speichern (`aktion_aendern`):** Baut aus den geänderten Eingabefeldern das aktualisierte Objekt, ruft `self.verwaltung.person_aktualisieren(obj)` sowie `self.verwaltung.speichern()` auf und aktualisiert die Tabelle.
5. **Löschen (`aktion_loeschen`):** Fragt per `messagebox.askyesno()` zur Sicherheit nach, löscht die Person über `person_loeschen(person_id)`, speichert die JSON-Datei und aktualisiert die Tabelle.

8.3 Aufgabe 7.3: Die vollständige Referenzimplementierung (Alle 3 Dateien)

Nachfolgend finden Sie den vollständig lauffähigen, dokumentierten Quellcode aller drei Projektdateien zum Abgleich und Testen Ihrer Lösung:

Listing 5: Datei 1: models.py (Abstrakte Basisklasse, Kindklassen, Overriding & Overloading)

```
from abc import ABC, abstractmethod


class Person(ABC):
    """Abstrakte Basisklasse für alle Angehörigen der Universität."""
```

```
6
7       def __init__(self, person_id: str, vorname: str, nachname: str, email: str):
8           if not str(person_id).strip():
9               raise ValueError("Die Personen-ID darf nicht leer sein!")
10          if not str(vorname).strip() or not str(nachname).strip():
11              raise ValueError("Vor- und Nachname dürfen nicht leer sein!")
12          if "@" not in str(email):
13              raise ValueError("Bitte eine gültige E-Mail-Adresse mit '@' eingeben!")
14
15          self.person_id = str(person_id).strip()
16          self.vorname = str(vorname).strip()
17          self.nachname = str(nachname).strip()
18          self.email = str(email).strip()
19
20      def voller_name(self) -> str:
21          return f"{self.vorname} {self.nachname}"
22
23      def erstelle_campus_ausweis(self) -> str:
24          """Standardmethode, die in Kindklassen überschrieben werden kann (Overriding)
                .."""
25          return f"AUSWEIS [{self.person_id}] {self.voller_name()} ({self.get_rolle()})
                "
26
27      @abstractmethod
28      def get_rolle(self) -> str:
29          pass
30
31      @abstractmethod
32      def get_details(self) -> str:
33          pass
34
35      @abstractmethod
36      def to_dict(self) -> dict:
37          pass
38
39      def __str__(self) -> str:
40          """Operator-Überladung für print() und str()."""
41          return f"[{self.person_id}] {self.voller_name()} ({self.get_rolle()}) | {self
                .get_details()}"
42
43      def __eq__(self, other) -> bool:
44          """Operator-Überladung für '==': Zwei Personen sind gleich bei identischer ID
                ."""
45          if not isinstance(other, Person):
46              return False
47          return self.person_id == other.person_id
48
49
50  class Studierender(Person):
51      """Konkrete Klasse für Studierende der Universität."""
52
53      def __init__(self, person_id: str, vorname: str, nachname: str, email: str,
54                   studiengang: str, semester: int = 1, ects: int = 0):
55          super().__init__(person_id, vorname, nachname, email)
56          if not str(studiengang).strip():
57              raise ValueError("Der Studiengang darf nicht leer sein!")
58          self.studiengang = str(studiengang).strip()
59          self.semester = int(semester)
60          self.ects = int(ects)
61
62      # Überschreiben (Method Overriding) der Ausweis-Methode
63      def erstelle_campus_ausweis(self) -> str:
64          return f"STUDIERENDENAUSWEIS [{self.person_id}] {self.voller_name()} | {self.
                studiengang} (inkl. NRW-Ticket)"
```

```python

    # Überladen (Method Overloading) durch Default-Parameter & Rückgabeweiche
    def studienfortschritt_berechnen(self, ziel_ects: int = 180, zeige_details: bool = False):
        prozent = round((self.ects / ziel_ects) * 100, 1) if ziel_ects > 0 else 0.0
        if zeige_details:
            return f"Fortschritt: {self.ects} / {ziel_ects} ECTS ({prozent}%)"
        return prozent

    def get_rolle(self) -> str:
        return "Studierender"

    def get_details(self) -> str:
        return f"Studiengang: {self.studiengang} (Sem. {self.semester}, {self.ects} ECTS)"

    def to_dict(self) -> dict:
        return {
            "rolle": self.get_rolle(),
            "person_id": self.person_id,
            "vorname": self.vorname,
            "nachname": self.nachname,
            "email": self.email,
            "studiengang": self.studiengang,
            "semester": self.semester,
            "ects": self.ects
        }


class Professor(Person):
    """Konkrete Klasse für Professorinnen und Professoren."""

    def __init__(self, person_id: str, vorname: str, nachname: str, email: str,
                 fachgebiet: str, vorlesungen=None):
        super().__init__(person_id, vorname, nachname, email)
        if not str(fachgebiet).strip():
            raise ValueError("Das Fachgebiet darf nicht leer sein!")
        self.fachgebiet = str(fachgebiet).strip()
        self.vorlesungen = []
        if vorlesungen:
            self.vorlesung_hinzufuegen(vorlesungen)

    # Überladen (Method Overloading) via *args und isinstance()
    def vorlesung_hinzufuegen(self, *neue_vorlesungen):
        """Akzeptiert Einzel-Strings, Listen von Strings oder mehrere Argumente (*args)."""
        for eintrag in neue_vorlesungen:
            if isinstance(eintrag, (list, tuple)):
                for einzel in eintrag:
                    titel = str(einzel).strip()
                    if titel and titel not in self.vorlesungen:
                        self.vorlesungen.append(titel)
            elif isinstance(eintrag, str):
                titel = eintrag.strip()
                if titel and titel not in self.vorlesungen:
                    self.vorlesungen.append(titel)

    # Überschreiben (Method Overriding) unter Einbezug von super()
    def erstelle_campus_ausweis(self) -> str:
        basis = super().erstelle_campus_ausweis()
        return f"{basis} - Zutritt: Alle Labore & Senatssaal"

    def get_rolle(self) -> str:
        return "Professor"
```

```
126
127     def get_details(self) -> str:
128         vl_str = ", ".join(self.vorlesungen) if self.vorlesungen else "Keine
                Vorlesungen"
129         return f"Lehrstuhl: {self.fachgebiet} | VL: {vl_str}"
130
131     def to_dict(self) -> dict:
132         return {
133             "rolle": self.get_rolle(),
134             "person_id": self.person_id,
135             "vorname": self.vorname,
136             "nachname": self.nachname,
137             "email": self.email,
138             "fachgebiet": self.fachgebiet,
139             "vorlesungen": self.vorlesungen
140         }
141
142
143 class Mitarbeiter(Person):
144     """Konkrete Klasse für wissenschaftliche und administrative Mitarbeiter (Staff).
            """
145
146     def __init__(self, person_id: str, vorname: str, nachname: str, email: str,
147                  abteilung: str, wochenstunden: float = 39.5):
148         super().__init__(person_id, vorname, nachname, email)
149         if not str(abteilung).strip():
150             raise ValueError("Die Abteilung darf nicht leer sein!")
151         stunden = float(wochenstunden)
152         if stunden <= 0 or stunden > 48:
153             raise ValueError("Wochenstunden müssen zwischen 0.5 und 48.0 liegen!")
154         self.abteilung = str(abteilung).strip()
155         self.wochenstunden = stunden
156
157     def get_rolle(self) -> str:
158         return "Mitarbeiter"
159
160     def get_details(self) -> str:
161         return f"Abteilung: {self.abteilung} ({self.wochenstunden:.1f} Std./Woche)"
162
163     def to_dict(self) -> dict:
164         return {
165             "rolle": self.get_rolle(),
166             "person_id": self.person_id,
167             "vorname": self.vorname,
168             "nachname": self.nachname,
169             "email": self.email,
170             "abteilung": self.abteilung,
171             "wochenstunden": self.wochenstunden
172         }
```

Listing 6: Datei 2: storage.py (Listenverwaltung und JSON-Persistenz)

```
1  import json
2  import os
3  from models import Person, Studierender, Professor, Mitarbeiter
4
5
6  class UniversitaetsVerwaltung:
7      """Verwaltet alle Personen in einer Liste und speichert/lädt diese als JSON."""
8
9      def __init__(self, dateipfad: str = "campus_daten.json"):
10         self.dateipfad = dateipfad
11         self.personen: list[Person] = []
12
```

```
    def finde_nach_id(self, person_id: str):
        for p in self.personen:
            if p.person_id == person_id:
                return p
        return None

    def person_hinzufuegen(self, neue_person: Person):
        if neue_person in self.personen:
            raise ValueError(f"Eine Person mit der ID '{neue_person.person_id}'
                existiert bereits!")
        self.personen.append(neue_person)

    def person_aktualisieren(self, aktualisierte_person: Person):
        for idx, p in enumerate(self.personen):
            if p.person_id == aktualisierte_person.person_id:
                self.personen[idx] = aktualisierte_person
                return
        raise KeyError(f"Person mit ID '{aktualisierte_person.person_id}' wurde nicht
             gefunden!")

    def person_loeschen(self, person_id: str) -> bool:
        gefunden = self.finde_nach_id(person_id)
        if gefunden:
            self.personen.remove(gefunden)
            return True
        return False

    def speichern(self):
        """Speichert die gesamte Personenliste als formatierte JSON-Datei."""
        daten = [p.to_dict() for p in self.personen]
        with open(self.dateipfad, "w", encoding="utf-8") as f:
            json.dump(daten, f, indent=4, ensure_ascii=False)

    @staticmethod
    def objekt_aus_dict(d: dict) -> Person:
        rolle = d.get("rolle", "")
        if rolle == "Studierender":
            return Studierender(
                d["person_id"], d["vorname"], d["nachname"], d["email"],
                d.get("studiengang", ""), d.get("semester", 1), d.get("ects", 0)
            )
        elif rolle == "Professor":
            return Professor(
                d["person_id"], d["vorname"], d["nachname"], d["email"],
                d.get("fachgebiet", ""), d.get("vorlesungen", [])
            )
        elif rolle == "Mitarbeiter":
            return Mitarbeiter(
                d["person_id"], d["vorname"], d["nachname"], d["email"],
                d.get("abteilung", ""), d.get("wochenstunden", 39.5)
            )
        raise ValueError(f"Unbekannte Rolle: {rolle}")

    def laden(self) -> bool:
        """Lädt die Personenliste bei Programmstart aus der JSON-Datei."""
        if not os.path.exists(self.dateipfad):
            return False
        try:
            with open(self.dateipfad, "r", encoding="utf-8") as f:
                roh_liste = json.load(f)
            self.personen = [self.objekt_aus_dict(d) for d in roh_liste]
            return True
        except Exception as e:
            print(f"[Fehler beim Laden]: {e}")
```

```
            return False

    def erzeuge_beispiel_daten(self):
        """Wird beim allerersten Start ausgeführt, falls noch keine JSON-Datei existiert."""
        self.personen = [
            Professor("P-101", "Peter", "Kern", "peter.kern@th-koeln.de",
                      "Digitalisierung & Coding", ["Coding und Digitalisierung", "Informatik 1"]),
            Professor("P-102", "Mirko", "Lotz-Blumberg", "mirko.lotz@th-koeln.de",
                      "Prozessanalyse & Projektarbeit", ["Projektmodul Coding", "Datenanalyse"]),
            Studierender("S-201", "Anna", "Schmidt", "anna.schmidt@smail.th-koeln.de",
                         "Maschinenbau", semester=3, ects=65),
            Studierender("S-202", "Lukas", "Meyer", "lukas.meyer@smail.th-koeln.de",
                         "Verfahrenstechnik", semester=5, ects=120),
            Mitarbeiter("M-301", "Sabine", "Köhler", "sabine.koehler@th-koeln.de",
                        "Prüfungsamt & Studierendenservice", wochenstunden=39.5)
        ]
        self.speichern()
```

Listing 7: Datei 3: gui_app.py (Tkinter-GUI zum Anlegen, Verändern, Löschen und Speichern)

```
import tkinter as tk
from tkinter import ttk, messagebox
from models import Studierender, Professor, Mitarbeiter
from storage import UniversitaetsVerwaltung


class CampusApp:
    """Grafische Tkinter-Anwendung zur Verwaltung von Personen an der Universität."""

    def __init__(self, root: tk.Tk):
        self.root = root
        self.root.title("TH-CampusAdmin - Objektorientierte Personenverwaltung")
        self.root.geometry("980x620")

        # 1. Backend-Verwaltung initialisieren & Daten bei Programmstart laden
        self.verwaltung = UniversitaetsVerwaltung("campus_daten.json")
        if not self.verwaltung.laden():
            self.verwaltung.erzeuge_beispiel_daten()

        self._baue_gui()
        self._aktualisiere_zusatzfelder()
        self.aktualisiere_tabelle()

        # Automatisches Speichern beim Schließen des Fensters sicherstellen
        self.root.protocol("WM_DELETE_WINDOW", self.beenden_und_speichern)

    def _baue_gui(self):
        # --- BEREICH 1: Eingabemaske (Oben) ---
        form_frame = ttk.LabelFrame(self.root, text=" Person anlegen oder bearbeiten ", padding=12)
        form_frame.pack(fill="x", padx=12, pady=8)

        # Zeile 0: Rolle & Personen-ID
        ttk.Label(form_frame, text="Rolle / Typ:").grid(row=0, column=0, sticky="w", pady=4)
        self.rolle_var = tk.StringVar(value="Studierender")
        self.combo_rolle = ttk.Combobox(
            form_frame, textvariable=self.rolle_var,
            values=["Studierender", "Professor", "Mitarbeiter"], state="readonly", width=22
```

```
        )
        self.combo_rolle.grid(row=0, column=1, sticky="w", padx=6, pady=4)
        self.combo_rolle.bind("<<ComboboxSelected>>", lambda e: self.
            _aktualisiere_zusatzfelder())

        ttk.Label(form_frame, text="Personen-ID:").grid(row=0, column=2, sticky="w",
            padx=(15, 0), pady=4)
        self.entry_id = ttk.Entry(form_frame, width=25)
        self.entry_id.grid(row=0, column=3, sticky="w", padx=6, pady=4)

        # Zeile 1: Vorname & Nachname
        ttk.Label(form_frame, text="Vorname:").grid(row=1, column=0, sticky="w", pady
            =4)
        self.entry_vorname = ttk.Entry(form_frame, width=25)
        self.entry_vorname.grid(row=1, column=1, sticky="w", padx=6, pady=4)

        ttk.Label(form_frame, text="Nachname:").grid(row=1, column=2, sticky="w",
            padx=(15, 0), pady=4)
        self.entry_nachname = ttk.Entry(form_frame, width=25)
        self.entry_nachname.grid(row=1, column=3, sticky="w", padx=6, pady=4)

        # Zeile 2: E-Mail & dynamisches Zusatzfeld 1
        ttk.Label(form_frame, text="E-Mail:").grid(row=2, column=0, sticky="w", pady
            =4)
        self.entry_email = ttk.Entry(form_frame, width=25)
        self.entry_email.grid(row=2, column=1, sticky="w", padx=6, pady=4)

        self.lbl_zusatz1 = ttk.Label(form_frame, text="Studiengang:")
        self.lbl_zusatz1.grid(row=2, column=2, sticky="w", padx=(15, 0), pady=4)
        self.entry_zusatz1 = ttk.Entry(form_frame, width=32)
        self.entry_zusatz1.grid(row=2, column=3, sticky="w", padx=6, pady=4)

        # Zeile 3: Dynamisches Zusatzfeld 2 & Zusatzfeld 3
        self.lbl_zusatz2 = ttk.Label(form_frame, text="Fachsemester:")
        self.lbl_zusatz2.grid(row=3, column=0, sticky="w", pady=4)
        self.entry_zusatz2 = ttk.Entry(form_frame, width=25)
        self.entry_zusatz2.grid(row=3, column=1, sticky="w", padx=6, pady=4)

        self.lbl_zusatz3 = ttk.Label(form_frame, text="ECTS-Punkte:")
        self.lbl_zusatz3.grid(row=3, column=2, sticky="w", padx=(15, 0), pady=4)
        self.entry_zusatz3 = ttk.Entry(form_frame, width=25)
        self.entry_zusatz3.grid(row=3, column=3, sticky="w", padx=6, pady=4)

        # --- BEREICH 2: Aktions-Buttons (Mitte) ---
        btn_frame = ttk.Frame(self.root, padding=6)
        btn_frame.pack(fill="x", padx=12)

        ttk.Button(btn_frame, text="Neu anlegen", command=self.aktion_anlegen).pack(
            side="left", padx=4)
        ttk.Button(btn_frame, text="Auswahl verändern (Update)", command=self.
            aktion_aendern).pack(side="left", padx=4)
        ttk.Button(btn_frame, text="Auswahl löschen", command=self.aktion_loeschen).
            pack(side="left", padx=4)
        ttk.Button(btn_frame, text="Maske leeren", command=self.maske_leeren).pack(
            side="left", padx=4)
        ttk.Button(btn_frame, text="Ausweis zeigen", command=self.aktion_ausweis).
            pack(side="left", padx=4)
        ttk.Button(btn_frame, text="Daten manuell speichern (JSON)", command=self.
            aktion_speichern).pack(side="right", padx=4)

        # --- BEREICH 3: Tabellenansicht (Unten) ---
        table_frame = ttk.LabelFrame(self.root, text=" Gespeicherte Personen der
            Universität ", padding=8)
        table_frame.pack(fill="both", expand=True, padx=12, pady=8)
```

```
90
91          spalten = ("id", "rolle", "name", "email", "details")
92          self.tree = ttk.Treeview(table_frame, columns=spalten, show="headings",
                height=12)
93          self.tree.heading("id", text="ID")
94          self.tree.heading("rolle", text="Rolle")
95          self.tree.heading("name", text="Voller Name")
96          self.tree.heading("email", text="E-Mail-Adresse")
97          self.tree.heading("details", text="Rollenspezifische Details (Polymorphismus)
                ")
98
99          self.tree.column("id", width=75, anchor="center")
100         self.tree.column("rolle", width=110, anchor="center")
101         self.tree.column("name", width=165)
102         self.tree.column("email", width=210)
103         self.tree.column("details", width=360)
104
105         scrollbar = ttk.Scrollbar(table_frame, orient="vertical", command=self.tree.
                yview)
106         self.tree.configure(yscrollcommand=scrollbar.set)
107         self.tree.pack(side="left", fill="both", expand=True)
108         scrollbar.pack(side="right", fill="y")
109
110         # Klick in Tabelle lädt Datensatz in die Bearbeitungsmaske
111         self.tree.bind("<<TreeviewSelect>>", self.on_tabellen_auswahl)
112
113         # Statuszeile am unteren Fensterrand
114         self.status_var = tk.StringVar(value="Bereit. Daten aus 'campus_daten.json'
                geladen.")
115         status_bar = ttk.Label(self.root, textvariable=self.status_var, relief="
                sunken", anchor="w", padding=4)
116         status_bar.pack(fill="x", side="bottom")
117
118     def _aktualisiere_zusatzfelder(self):
119         """Passt die Beschriftung der Zusatzfelder 1-3 dynamisch an die gewählte
                Rolle an."""
120         rolle = self.rolle_var.get()
121         if rolle == "Studierender":
122             self.lbl_zusatz1.config(text="Studiengang:")
123             self.lbl_zusatz2.config(text="Fachsemester:")
124             self.lbl_zusatz3.config(text="ECTS-Punkte:")
125             self.entry_zusatz3.config(state="normal")
126         elif rolle == "Professor":
127             self.lbl_zusatz1.config(text="Fachgebiet / Lehrstuhl:")
128             self.lbl_zusatz2.config(text="Vorlesungen (mit Komma):")
129             self.lbl_zusatz3.config(text="(Nicht benötigt):")
130             self.entry_zusatz3.delete(0, tk.END)
131             self.entry_zusatz3.config(state="disabled")
132         elif rolle == "Mitarbeiter":
133             self.lbl_zusatz1.config(text="Abteilung:")
134             self.lbl_zusatz2.config(text="Wochenstunden:")
135             self.lbl_zusatz3.config(text="(Nicht benötigt):")
136             self.entry_zusatz3.delete(0, tk.END)
137             self.entry_zusatz3.config(state="disabled")
138
139     def _baue_objekt_aus_maske(self):
140         """Liest alle Eingabefelder aus und erzeugt das passende Personen-Objekt."""
141         rolle = self.rolle_var.get()
142         p_id = self.entry_id.get().strip()
143         vorname = self.entry_vorname.get().strip()
144         nachname = self.entry_nachname.get().strip()
145         email = self.entry_email.get().strip()
146         z1 = self.entry_zusatz1.get().strip()
147         z2 = self.entry_zusatz2.get().strip()
```

```
148         z3 = self.entry_zusatz3.get().strip()
149 
150         if rolle == "Studierender":
151             sem = int(z2) if z2 else 1
152             ects = int(z3) if z3 else 0
153             return Studierender(p_id, vorname, nachname, email, studiengang=z1,
                    semester=sem, ects=ects)
154         elif rolle == "Professor":
155             prof = Professor(p_id, vorname, nachname, email, fachgebiet=z1)
156             # Nutzt unsere überladene Methode vorlesung_hinzufuegen mit einer Liste
157             vl_liste = [v.strip() for v in z2.split(",") if v.strip()]
158             prof.vorlesung_hinzufuegen(vl_liste)
159             return prof
160         elif rolle == "Mitarbeiter":
161             stunden = float(z2.replace(",", ".")) if z2 else 39.5
162             return Mitarbeiter(p_id, vorname, nachname, email, abteilung=z1,
                    wochenstunden=stunden)
163 
164     def aktualisiere_tabelle(self):
165         """Leert die Treeview-Tabelle und füllt sie frisch aus self.verwaltung.
                personen."""
166         for zeile in self.tree.get_children():
167             self.tree.delete(zeile)
168         for p in self.verwaltung.personen:
169             self.tree.insert("", tk.END, values=(
170                 p.person_id, p.get_rolle(), p.voller_name(), p.email, p.get_details()
171             ))
172         self.status_var.set(f"Gesamtanzahl gespeicherter Personen: {len(self.
                verwaltung.personen)}")
173 
174     def maske_leeren(self):
175         for entry in (self.entry_id, self.entry_vorname, self.entry_nachname,
176                       self.entry_email, self.entry_zusatz1, self.entry_zusatz2):
177             entry.delete(0, tk.END)
178         self.entry_zusatz3.config(state="normal")
179         self.entry_zusatz3.delete(0, tk.END)
180         self._aktualisiere_zusatzfelder()
181         self.tree.selection_remove(self.tree.selection())
182 
183     def on_tabellen_auswahl(self, event):
184         """Befüllt die Eingabemaske, sobald eine Person in der Tabelle angeklickt
                wird."""
185         auswahl = self.tree.selection()
186         if not auswahl:
187             return
188         werte = self.tree.item(auswahl[0], "values")
189         person_id = werte[0]
190         p = self.verwaltung.finde_nach_id(person_id)
191         if not p:
192             return
193 
194         self.rolle_var.set(p.get_rolle())
195         self._aktualisiere_zusatzfelder()
196 
197         self.entry_id.delete(0, tk.END)
198         self.entry_id.insert(0, p.person_id)
199         self.entry_vorname.delete(0, tk.END)
200         self.entry_vorname.insert(0, p.vorname)
201         self.entry_nachname.delete(0, tk.END)
202         self.entry_nachname.insert(0, p.nachname)
203         self.entry_email.delete(0, tk.END)
204         self.entry_email.insert(0, p.email)
205 
206         self.entry_zusatz1.delete(0, tk.END)
```

```
207         self.entry_zusatz2.delete(0, tk.END)
208         self.entry_zusatz3.config(state="normal")
209         self.entry_zusatz3.delete(0, tk.END)
210 
211         if isinstance(p, Studierender):
212             self.entry_zusatz1.insert(0, p.studiengang)
213             self.entry_zusatz2.insert(0, str(p.semester))
214             self.entry_zusatz3.insert(0, str(p.ects))
215         elif isinstance(p, Professor):
216             self.entry_zusatz1.insert(0, p.fachgebiet)
217             self.entry_zusatz2.insert(0, ", ".join(p.vorlesungen))
218             self.entry_zusatz3.config(state="disabled")
219         elif isinstance(p, Mitarbeiter):
220             self.entry_zusatz1.insert(0, p.abteilung)
221             self.entry_zusatz2.insert(0, str(p.wochenstunden))
222             self.entry_zusatz3.config(state="disabled")
223 
224     def aktion_anlegen(self):
225         try:
226             neue_person = self._baue_objekt_aus_maske()
227             self.verwaltung.person_hinzufuegen(neue_person)
228             self.verwaltung.speichern()
229             self.aktualisiere_tabelle()
230             self.maske_leeren()
231             self.status_var.set(f"Person '{neue_person.voller_name()}' erfolgreich
                    angelegt und gespeichert.")
232         except ValueError as e:
233             messagebox.showerror("Fehler beim Anlegen", str(e))
234 
235     def aktion_aendern(self):
236         try:
237             akt_person = self._baue_objekt_aus_maske()
238             self.verwaltung.person_aktualisieren(akt_person)
239             self.verwaltung.speichern()
240             self.aktualisiere_tabelle()
241             self.status_var.set(f"Person '{akt_person.person_id}' erfolgreich
                    aktualisiert und gespeichert.")
242         except (ValueError, KeyError) as e:
243             messagebox.showerror("Fehler beim Ändern", str(e))
244 
245     def aktion_loeschen(self):
246         p_id = self.entry_id.get().strip()
247         if not p_id:
248             messagebox.showwarning("Hinweis", "Bitte wählen Sie zuerst eine Person in
                    der Tabelle aus.")
249             return
250         if messagebox.askyesno("Löschen bestätigen", f"Möchten Sie die Person mit ID
                '{p_id}' wirklich löschen?"):
251             if self.verwaltung.person_loeschen(p_id):
252                 self.verwaltung.speichern()
253                 self.aktualisiere_tabelle()
254                 self.maske_leeren()
255                 self.status_var.set(f"Person '{p_id}' gelöscht und JSON-Datei
                        aktualisiert.")
256             else:
257                 messagebox.showerror("Fehler", f"Keine Person mit ID '{p_id}'
                        gefunden.")
258 
259     def aktion_ausweis(self):
260         """Demonstriert das Methoden-Überschreiben (erstelle_campus_ausweis) per
                Dialogfenster."""
261         p_id = self.entry_id.get().strip()
262         p = self.verwaltung.finde_nach_id(p_id)
263         if not p:
```

```
264             messagebox.showinfo("Ausweis-Vorschau", "Bitte wählen Sie zuerst eine
                    Person in der Tabelle aus!")
265             return
266         messagebox.showinfo("Offizieller Campus-Ausweis", p.erstelle_campus_ausweis()
                )
267
268     def aktion_speichern(self):
269         self.verwaltung.speichern()
270         messagebox.showinfo("Gespeichert", f"Alle {len(self.verwaltung.personen)}
                Personen wurden in '{self.verwaltung.dateipfad}' gesichert.")
271
272     def beenden_und_speichern(self):
273         self.verwaltung.speichern()
274         self.root.destroy()
275
276
277 if __name__ == "__main__":
278     hauptfenster = tk.Tk()
279     app = CampusApp(hauptfenster)
280     hauptfenster.mainloop()
```

9 Modul 8: Erweiterungsaufgaben für Fortgeschrittene & Reflexion

Wenn Ihre Anwendung läuft und Sie bereits Personen über die Tkinter-Maske anlegen, verändern, löschen und nach einem Neustart wiederfinden können, erweitern Sie das System um zwei praxisnahe Zusatzfunktionen:

9.1 Aufgabe 8.1: Such- und Filterfunktion in der GUI ergänzen

Fügen Sie oberhalb der Tabelle ein kleines Suchfeld (`ttk.Entry`) hinzu. Sobald der Benutzer dort einen Suchbegriff eingibt (z. B. `"Maschinenbau"` oder `"Kern"`), soll die Tabelle nur noch diejenigen Personen aus `self.verwaltung.personen` anzeigen, in deren String-Darstellung (`str(p).lower()`) der Suchbegriff vorkommt.

9.2 Aufgabe 8.2: Eine vierte Kindklasse `Lehrbeauftragter` ableiten

Testen Sie, wie leicht sich Ihre objektorientierte Architektur erweitern lässt:

1. Leiten Sie in `models.py` eine neue Klasse `class Lehrbeauftragter(Professor):` direkt von `Professor` ab (zweistufige Vererbung!).
2. Ergänzen Sie das zusätzliche Attribut `stundensatz: float` (Honorar pro Lehrstunde) und überschreiben Sie `get_rolle(self)`, `get_details(self)` sowie `to_dict(self)`.
3. Prüfen Sie, wie wenige Zeilen Sie in `storage.py` anpassen müssen, damit auch Lehrbeauftragte dauerhaft gespeichert und geladen werden können.

9.3 Abschluss-Kontrollfragen zu Modul 7 & 8

1. Was bewirkt die Zeile `self.root.protocol("WM_DELETE_WINDOW", self.beenden_und_speichern)` im Konstruktor unserer GUI-Klasse?
2. Warum haben wir die Datenhaltung (`UniversitaetsVerwaltung` in `storage.py`) und die grafische Oberfläche (`CampusApp` in `gui_app.py`) in zwei getrennte Klassen aufgeteilt? Wie nennt man dieses Architekturprinzip in der Softwareentwicklung?
3. Reflektieren Sie alle Konzepte dieses Übungsblatts: An welchen konkreten Code-Stellen haben wir **Abstraktion**, **Vererbung**, **Überschreiben (Overriding)**, **Überladen (Overloading)** und **Polymorphismus** eingesetzt?

Herzlichen Glückwunsch!

Sie haben eine vollständige, mehrschichtige Desktop-Anwendung mit abstrakter Klassenhierarchie, Tkinter-Oberfläche und JSON-Persistenz in Python entwickelt!